

Plan d'Organisation Général du Système

[logiciel_V4]

- └─  Documentation (Manuels, Références, Specs)
- └─  Moteur (Le cœur technologique)
- | └─  Backend (Serveur FastAPI Python & Algorithmes)
- | └─  Frontend (Interface Graphique React & Vite)
- └─  hmenuiserie (Module connexe d'atelier mobile / QR Scanner)

PARTIE 1 : L'Arborescence Complète des Fichiers (Où est quoi ?)

L'architecture suit un modèle **Decoupled Desktop Edge** (SaaS local découplé). Voici comment sont rangés les composants clés :

1. Le Dossier Central de Documentation (/Documentation)

C'est la mémoire technique du projet. Il contient les fichiers maîtres :

- ARCHITECTURE_DETAILLEE.md : Cartographie complète de l'application.
- MANUEL_INGENIERIE.md : Théorie mathématique, structures géométriques et algorithmes.
- REFERENCE_TECHNIQUE.md : Guide pratique pour comprendre la stack et les piliers technologiques.
- API_REFERENCE.md & BACKEND_API_REQUIREMENTS.md : Spécifications des routes de communication HTTP.
- MODULE_SAUVEGARDE_RESTAURATION.md : Fonctionnement de la sécurité des données.
- TESTING_GUIDE.md : Protocoles de tests et de débogage.

2. Le Backend / Moteur de calcul (/Moteur/Backend)

Divisé en deux grandes sous-structures : la tuyauterie réseau (System/Bin) et l'intelligence de calcul (Services/IA_Engine).

- **System/Bin/app/ (La structure de l'application Web locale)**
 - main.py : Point d'entrée du serveur FastAPI, configure les permissions (CORS ouverts pour le dev).
 - models/ : Modèles de la base de données SQLite (Schémas relationnels).
 - schemas/ : Validation des données entrantes/sortantes via *Pydantic* (ex: structure d'une requête OptimizationRequest).
 - routers/ : Les aiguilleurs qui reçoivent les requêtes de l'interface :
 - projects.py (Statistiques et agrégations)
 - materials.py (Identification et calcul des coûts/poids)
 - stock.py (Disponibilité instantanée)

- optimize.py (Lancement du calcul)
- **Services/IA_Engine/ (Le cerveau mathématique)**
 - advanced_optimizer.py : Script maître de découpe orthogonale (panneaux).
 - exports.py : Module de génération de rapports (PDF, images des plans de coupe).
 - step_parser.py : Analyseur géométrique 3D pour importer des fichiers de CAO industrielle (.step).
 - raw_wood_optimizer/core.py : Algorithmes spécialisés dans le bois massif, les formes irrégulières et la détection de défauts.

3. Le Frontend / L'Interface Utilisateur (/Moteur/Frontend)

Développé en **React 18**, **TypeScript** et **TailwindCSS**.

- src/components/Optimize/ : Les éléments de l'écran principal :
 - EnhancedProjectSelector.tsx : Sélection et statistiques des projets.
 - MaterialBreakdown.tsx : Tableau d'analyse des matériaux requis.
 - MaterialSourceSelector.tsx : Boutons pour choisir entre "Puiser dans le Stock" ou "Acheter chez le Fournisseur".
 - PieceSelector.tsx & StockSelector.tsx : Listes de sélection avancées.
- src/services/ : Modules JavaScript (projectService.ts, optimizeService.ts, etc.) chargés d'appeler le backend via des requêtes asynchrones.
- src/pages/Optimize.tsx : La page maîtresse qui assemble tous les composants.

4. Le Module Atelier (/hmenuiserie)

Une application Web mobile d'appoint (avec serveur server.js, app.js, manifest.json pour fonctionnement en PWA offline) intégrant :

- qrScanner.js : Outil pour scanner des QR codes à l'atelier afin de suivre et synchroniser l'utilisation des panneaux ou des chutes en temps réel (syncQueue.js).

PARTIE 2 : Comment fonctionnent les fonctionnalités (Les Flux Logiques)

Voici le cycle de vie informatique complet d'une session de travail dans OptiCut Pro V4, de la sélection à la découpe.

1. La Phase de Préparation (Collecte et Analyse)

Lorsque l'utilisateur charge des projets dans l'interface, un ensemble de requêtes asynchrones s'active en arrière-plan :

[UI React] —(1) Sélection Projets—> [FastAPI: /api/projects/{id}/stats] —> (SQL Count/Sum) —> [Base SQLite]

[UI React] <—(2) Retourne: Qté pièces, matériaux, surface totale (mm²)

1. **Identification et conversion (/api/materials/identify-from-pieces)** : Le backend reçoit les identifiants des pièces. Il va chercher l'essence du bois (chêne, hêtre, sapin...) et l'épaisseur dans la base SQLite.
 - *Calcul de masse* : Il convertit la surface cumulée des pièces de mm^2 en m^2 . En fonction de l'épaisseur et de la densité du bois (ex: 600 kg/ m^3 pour le MDF, 700 kg/ m^3 pour le chêne), il **estime informatiquement le poids total en kg** de la commande.
 - *Calcul financier* : Il applique le type de tarification enregistré (au m^2 , au m^3 , ou à l'unité) pour sortir une estimation financière immédiate.
2. **Vérification des Ressources (/api/stock/availability)** : Le système scanne la table Stock de la base de données. Il isole les panneaux correspondants aux matériaux demandés, filtre ceux dont la quantité est égale à 0, et affiche les panneaux utilisables directement dans l'interface.

2. Le Moteur de Calcul Géométrique et Mathématique (advanced_optimizer.py)

Une fois que l'utilisateur valide ses choix et ses sources (Stock ou Fournisseur), le backend exécute le moteur d'optimisation (Problème mathématique du *2D Bin Packing*). Il utilise un modèle de conception informatique dynamique (**Pattern Strategy**) :

Cas A : Optimisation de Panneaux (Découpe Rectangulaire Standard)

Le logiciel déploie des algorithmes de recherche opérationnelle rapides :

- **L'algorithme Guillotine (Split Strategy ADAPTIVE)** : Il simule informatiquement des coupes droites d'un bord à l'autre d'un panneau (comme une scie industrielle). À chaque pièce placée, la zone restante est subdivisée en nouvelles structures rectangulaires de tailles plus petites.
- **L'algorithme Skyline++** : Il modélise le panneau en gérant une structure de données en "ligne d'horizon" (*Skyline*). Il cherche à imbriquer les pièces au point le plus bas pour caler au maximum les composants dans les angles morts.

Cas B : Optimisation de Bois Brut et Nesting (raw_wood_optimizer)

Pour les formes non standard et le bois massif, le logiciel bascule sur de la géométrie polygonale computationnelle :

- **Algorithme NFP (No-Fit Polygon)** : Il calcule mathématiquement les frontières de collision entre deux polygones complexes pour les emboîter parfaitement sans chevauchement. Si le calcul est trop lourd, un système de repli (*fallback*) englobe la forme dans un rectangle virtuel simplifié (*Bounding Box*).
- **Contraintes physiques de matrice** :
 - **Le Kerf** : C'est l'épaisseur de la lame (ex: 3 mm). Le logiciel ajoute informatiquement la valeur du kerf autour de chaque pièce dans sa matrice tridimensionnelle pour que la perte de bois brûlé par la scie soit anticipée.
 - **Le Grain Vector** : Si le sens du fil du bois est activé (grain_strict), la matrice logicielle interdit l'application de rotations à 90° sur la pièce. Elle bloque l'orientation à 0° ou 180° pour assurer l'esthétique et la solidité de la pièce finale.

Évaluation et Arbitrage (K-Metric & CP-SAT)

Chaque solution générée reçoit une note de performance appelée **K-Metric** (Indice de Kenyon), un nombre flottant compris entre 0.0 (perte totale) et 1.0 (aucun reste).

- Si le rendement calculé est insuffisant (inférieur à **85%**), et si le nombre de pièces reste modéré, le logiciel appelle le solveur **Google OR-Tools (CP-SAT)**. Ce moteur de programmation par contraintes mathématiques exactes tourne de manière intensive pendant un temps imparti maximum (`time_limit_seconds`) pour forcer l'agencement le plus parfait possible.

PARTIE 3 : Robustesse et Systèmes de Sécurité Industrielle

Conçu pour tourner en environnement d'atelier (où les coupures de courant ou fermetures brusques arrivent), le logiciel intègre deux modules de bas niveau permanents :

1. La Télémétrie en Arrière-Plan (*Sidecar Log Server*)

Pendant que le logiciel tourne, un mini-serveur TCP indépendant écoute sur le **port 9999**. C'est un outil d'audit. Il surveille le "battement de cœur" du processeur, la charge système et la température. Si le backend FastAPI subit un crash à cause d'une surcharge de calcul mathématique, le module *Sidecar* capture et fige les journaux d'erreurs (*logs*) de manière isolée pour éviter toute perte de diagnostic.

2. Le Script de Sauvegarde Automatique et Rétention (`auto_backup_shutdown.py`)

Ce module utilise des écouteurs de signaux système profonds (`atexit`, `SIGINT`, `SIGTERM`). Dès que l'utilisateur quitte le logiciel, ferme sa console ou fait un `Ctrl + C`, la séquence de sécurité industrielle suivante s'exécute automatiquement :

[Signal Fermeture / `Ctrl+C`]

|
▼

1. Base SQLite figée (Plus d'écriture)

|
▼

2. Calcul du Hash SHA256 (Empreinte numérique d'intégrité) -> Écrit dans `metadata.json`

|
▼

3. Compression ZIP (Algorithme `ZIP_DEFLATED` - Niveau 9 maximum)

Contenu : Base de données (`opticut.db`) + Fiches exports + `metadata.json`

|
▼

4. Application de la règle de rétention :

- Conserve TOUJOURS les 5 dernières sauvegardes (ex: backup_2026-06-08_22-15.zip)
- Isole et conserve 1 sauvegarde mensuelle d'archive
- Supprime automatiquement les fichiers périmés pour vider le disque dur